MeatTrace: Python proof of concept

Lucas Clodic, Elisa Ouillet, Julia Kozhina

0. Tools we use

- `hashlib` computes SHA-256 fingerprints
- `json` turns a Python dictionary into stable text, so the same record always gives the same fingerprint

```
import hashlib
import json
```

1. Demonstration A: data integrity

1.1 One business transaction as a Python dictionary

The abattoir records that batch `RO-2026-0412` is horse meat, 2,400 kg. A dictionary stores pairs `key: value`, like the fields of a form.

```
slaughter_record = {
    "record_id": "REC-001",
    "event": "SLAUGHTER",
    "batch_id": "RO-2026-0412",
    "species": "HORSE",
    "weight_kg": 2400,
    "site": "Carpati Meat (RO)",
    "date": "2026-10-05",
}

print(slaughter_record)
```

```
{'record_id': 'REC-001', 'event': 'SLAUGHTER', 'batch_id': 'RO-2026-0412', 'species': 'HORSE', 'weight_kg': 2400,
```

1.2 Stable text, then a SHA-256 fingerprint

`json.dumps(record, sort_keys=True)` writes the fields in alphabetical order. The same record therefore always produces the same text, and the same fingerprint. `hashlib.sha256(...).hexdigest()` returns a 64-character fingerprint.

```
def record_hash(record):
    text = json.dumps(record, sort_keys=True)          # stable text
    return hashlib.sha256(text.encode()).hexdigest()   # SHA-256 fingerprint

print("Stable text :", json.dumps(slaughter_record, sort_keys=True))
print()
approved_hash = record_hash(slaughter_record)
print("Approved hash:", approved_hash)
```

```
Stable text : {"batch_id": "RO-2026-0412", "date": "2026-10-05", "event": "SLAUGHTER", "record_id": "REC-001", "s

Approved hash: b2d7a81a41a3ae8e3847b21c821538803f7268b85e502cb86736d78f6d2272e6
```

1.3 The 2013 fraud: the species changes from HORSE to BEEF

We do not modify the original record. We build a **new** dictionary with one different field: `{**slaughter_record, "species": "BEEF"}` copies every field, then replaces `species`.

```
relabelled_record = {**slaughter_record, "species": "BEEF"}   # new dict, original unchanged
relabelled_hash = record_hash(relabelled_record)

print("Original species  :", slaughter_record["species"])
print("Relabelled species:", relabelled_record["species"])
print("Approved hash  :", approved_hash)
print("Relabelled hash:", relabelled_hash)
print("Same record?   :", approved_hash == relabelled_hash)
```

```
Original species  : HORSE
Relabelled species: BEEF

Approved hash  : b2d7a81a41a3ae8e3847b21c821538803f7268b85e502cb86736d78f6d2272e6
Relabelled hash: b2c60303ead9a020cc82bbf4de7292c4885478ab67f2ec54c967d8992228bc52
Same record?   : False
```

1.4 Business interpretation

- One changed word gives a **completely different** fingerprint.
- In MeatTrace, every validator stores the approved fingerprint. If a trader relabels the batch, the next partner or an auditor recomputes the fingerprint, sees that it no longer matches, and blocks the batch.
- **Limit:** the hash proves that the record was not changed after approval. It does not prove that the abattoir told the truth. That is the job of the independent DNA test.

2. Extension: a block linked to the previous block

A block groups the fingerprints of several transactions and stores the hash of the previous block (`previous_hash`). Here, block 41 contains three transactions for the same batch: slaughter, transfer and DNA test.

```
transfer_record = {
    "record_id": "REC-002",
    "event": "TRANSFER",
    "batch_id": "RO-2026-0412",
    "from": "Carpati Meat (RO)",
    "to": "Delta Trading (NL)",
    "species_declared": "HORSE",
    "weight_kg": 2400,
}

dna_test_record = {
    "record_id": "REC-003",
    "event": "DNA_TEST",
    "batch_id": "RO-2026-0412",
    "lab": "GenoLab (DE)",
    "result": "EQUINE 100%",
    "report_sha256": record_hash({"report": "GL-2026-881.pdf"}),  # full report stays off-chain
}

previous_block_hash = record_hash({"block": 40, "note": "previous block"})

block_41 = {
    "block_number": 41,
    "timestamp": "2026-10-06T09:14:00Z",
    "previous_hash": previous_block_hash,
    "transactions": [record_hash(r) for r in (slaughter_record, transfer_record, dna_test_record)],
}
block_41_hash = record_hash(block_41)

print("Block 40 hash          :", previous_block_hash)
print("Block 41 previous_hash :", block_41["previous_hash"])
print("Block 41 transactions  :")
for tx in block_41["transactions"]:
    print("   ", tx)
print("Block 41 hash          :", block_41_hash)
```

```
Block 40 hash          : 29726fadb781f5e3f582c80bf3b7a9853f120274ae4056194bbdfb2de52b42de
Block 41 previous_hash : 29726fadb781f5e3f582c80bf3b7a9853f120274ae4056194bbdfb2de52b42de
Block 41 transactions  :
    b2d7a81a41a3ae8e3847b21c821538803f7268b85e502cb86736d78f6d2272e6
    e8ecd7fed07e7885a854863dd1902b638ec2ff893f4c4b7243230ab996ea60b7
    0a98d81b19ff396b8ab67722908ffb60d1f728ffa496352ad5ebd2de0ea505a1
Block 41 hash          : 9c2833fa8fa111e44e17a33253067832dd7d4a09f36cf2a39348b233a0a3b30c
```

2.1 Someone rewrites an old record

We rebuild block 41 with the relabelled slaughter record (BEEF) and recompute its hash.

```
tampered_block = {
    **block_41,
    "transactions": [record_hash(relabelled_record)] + block_41["transactions"][1:],
}
tampered_block_hash = record_hash(tampered_block)

print("Block 41 hash, approved          :", block_41_hash)
```

```
print("Block 41 hash after relabelling  :", tampered_block_hash)
print("Block still valid?               :", tampered_block_hash == block_41_hash)
```

```
Block 41 hash, approved          : 9c2833fa8fa111e44e17a33253067832dd7d4a09f36cf2a39348b233a0a3b30c
Block 41 hash after relabelling  : b992d559746882244f36a7f6ddf7c8041ba154a66ef5ee704112f7726862934f
Block still valid?               : False
```

Interpretation. Changing one old record changes the block hash. Block 42 stores the hash of block 41, so the chain breaks, and every member who holds a copy can see it.

2.2 The real 2013 scenario: a new document with another species

In 2013 nobody rewrote the abattoir's record. A trader issued a **new** sales document saying "beef". The hash test alone does not catch this, because the new document has its own valid fingerprint. MeatTrace adds a business rule: **the species declared in a transfer must equal the species signed in the batch history.** If not, the batch is blocked before it reaches FreshMeals.

```
resale_transfer = {
    **transfer_record,
    "record_id": "REC-004",
    "from": "Delta Trading (NL)",
    "to": "FreshMeals (FR)",
    "species_declared": "BEEF",
}
signed_species = slaughter_record["species"]
status = "OK" if resale_transfer["species_declared"] == signed_species else "MISMATCH, batch blocked"

print("REC-004 declares       :", resale_transfer["species_declared"])
print("Signed history says    :", signed_species)
print("Check                  :", status)
```

```
REC-004 declares       : BEEF
Signed history says    : HORSE
Check                  : MISMATCH, batch blocked
```

Interpretation. The relabelled batch is stopped at the next handover. The trader who signed REC-004 is identified by its signature, so liability is clear. The rule still relies on the abattoir telling the truth at the start: random DNA tests check that.

3. Demonstration B: access rights

3.1 Users and roles

Six users from five organizations. Each user has one role and an `active` flag (`False` means the account was revoked).

```
users = {
    "ion":    {"org": "Carpati Meat (RO)",     "role": "abattoir",        "active": True},
    "pieter": {"org": "Delta Trading (NL)",    "role": "trader",          "active": True},
    "dan":    {"org": "Delta Trading (NL)",    "role": "trader",          "active": False},
    "claire": {"org": "FreshMeals (FR)",       "role": "quality_manager", "active": True},
    "anna":   {"org": "GenoLab (DE)",          "role": "lab_inspector",   "active": True},
    "marc":   {"org": "Food Safety Authority", "role": "auditor",         "active": True},
}

for name, account in users.items():
    status = "active" if account["active"] else "REVOKED"
    print(f"{name:<7} {account['org']:<22} {account['role']:<16} {status}")
```

```
ion     Carpati Meat (RO)      abattoir         active
pieter  Delta Trading (NL)     trader           active
dan     Delta Trading (NL)     trader           REVOKED
claire  FreshMeals (FR)        quality_manager  active
anna    GenoLab (DE)           lab_inspector    active
marc    Food Safety Authority  auditor          active
```

3.2 The permission matrix

Management decides which role may perform each action. Python only applies these decisions. No role has `edit_record`: an approved record is never edited, it can only be corrected by a new, signed record.

```
role_permissions = {
    "abattoir":        {"create_record", "read_own"},
    "trader":          {"create_record", "read_own"},
    "quality_manager": {"create_record", "approve_record", "read_batch"},
    "lab_inspector":   {"create_test", "approve_record", "read_batch"},
```

```
    "auditor":         {"read_audit"},
}

records = {
    "REC-001": {"event": "SLAUGHTER", "created_by": "ion"},
    "REC-007": {"event": "RECEPTION", "created_by": "claire"},
}

for role, permissions in role_permissions.items():
    print(f"{role:<16} {sorted(permissions)}")
```

```
abattoir         ['create_record', 'read_own']
trader           ['create_record', 'read_own']
quality_manager  ['approve_record', 'create_record', 'read_batch']
lab_inspector    ['approve_record', 'create_test', 'read_batch']
auditor          ['read_audit']
```

3.3 The decision function `can_do`

The function checks three rules, in order:

1. **Revocation:** a revoked account can do nothing.
2. **Role-based access control:** the action must be in the role's permissions.
3. **Separation of duties:** nobody approves a record they created.

```
def can_do(user, action, record_id=None):
    account = users[user]
    if not account["active"]:
        return False, "account revoked"
    if action not in role_permissions[account["role"]]:
        return False, f"role '{account['role']}' has no '{action}' right"
    if action == "approve_record" and records[record_id]["created_by"] == user:
        return False, "separation of duties: cannot approve own record"
    return True, "permission granted"
```

3.4 Five tests: two allowed, three denied

```
tests = [
    ("anna",   "approve_record", "REC-001"),
    ("claire", "read_batch",     "REC-001"),
    ("pieter", "edit_record",    "REC-001"),
    ("claire", "approve_record", "REC-007"),
    ("dan",    "create_record",  None),
]

results = []
for user, action, record_id in tests:
    allowed, reason = can_do(user, action, record_id)
    results.append({
        "Result": "ALLOWED" if allowed else "DENIED",
        "User": user,
        "Role": users[user]["role"],
        "Action": action,
        "Record": record_id or "—",
        "Reason": reason,
    })

try:
    import pandas as pd
    display(pd.DataFrame(results))
except ImportError:
    for row in results:
        print(" | ".join(str(value) for value in row.values()))
```

	Result	User	Role	Action	Record	Reason
0	ALLOWED	anna	lab_inspector	approve_record	REC-001	permission granted
1	ALLOWED	claire	quality_manager	read_batch	REC-001	permission granted
2	DENIED	pieter	trader	edit_record	REC-001	role 'trader' has no 'edit_record' right
3	DENIED	claire	quality_manager	approve_record	REC-007	separation of duties: cannot approve own record
4	DENIED	dan	trader	create_record	-	account revoked

3.5 Business interpretation

Test	Result	Business risk handled
anna (DNA lab) approves the abattoir's record REC-001	ALLOWED	An independent party checks the record before it is accepted
claire (FreshMeals) reads the batch history	ALLOWED	The buyer checks the origin before cooking
pieter (trader) edits REC-001	DENIED	Stops relabelling horse as beef, the 2013 fraud
claire approves her own reception REC-007	DENIED	Separation of duties: no self-certification
dan, who left the trader, creates a record	DENIED	An ex-employee cannot sign in a partner's name

4. Optional extension (a): digital signature

Optional, not required by the brief. As in session 3, the abattoir signs its record with its **private key**; any partner checks the signature with the **public key**. This proves *who* issued the record and that it was *not changed*. It needs the `rsa` library (`pip install rsa`).

```
try:
    import rsa
except ImportError:
    rsa = None
    print("The 'rsa' library is missing. Install it with: pip install rsa")

if rsa:
    abattoir_public_key, abattoir_private_key = rsa.newkeys(1024)

    def stable_bytes(record):
        return json.dumps(record, sort_keys=True).encode()

    def verify(record, signature, public_key):
        try:
            rsa.verify(stable_bytes(record), signature, public_key)
            return "VALID SIGNATURE"
        except rsa.VerificationError:
            return "INVALID SIGNATURE"

    signature = rsa.sign(stable_bytes(slaughter_record), abattoir_private_key, "SHA-256")
    print("Signature length:", len(signature), "bytes")
    print("Original record (HORSE)  :", verify(slaughter_record, signature, abattoir_public_key))
    print("Relabelled record (BEEF) :", verify(relabelled_record, signature, abattoir_public_key))
```

```
Signature length: 128 bytes
Original record (HORSE)  : VALID SIGNATURE
Relabelled record (BEEF) : INVALID SIGNATURE
```

Interpretation. The abattoir's signature is valid only for the record it really signed. A relabelled record with the old signature is rejected, so nobody can claim the abattoir declared "BEEF".

5. Optional extension (b): consensus, 3 of 4 validators

Optional. Four known organizations validate blocks: FreshMeals, Retailer A, Retailer B and GenoLab. A validator approves a block only if its `previous_hash` matches the last block it holds and every transaction is one it received signed. The block is **accepted when at least 3 of 4 validators approve**.

```
original_transactions = set(block_41["transactions"])

validators = {
    "FreshMeals": {"online": True, "known_transactions": original_transactions},
    "Retailer A": {"online": True, "known_transactions": original_transactions},
    "Retailer B": {"online": True, "known_transactions": original_transactions},
    "GenoLab":    {"online": True, "known_transactions": original_transactions},
}

def validator_approves(validator, block):
    links_ok = block["previous_hash"] == previous_block_hash
    transactions_ok = all(tx in validator["known_transactions"] for tx in block["transactions"])
    return validator["online"] and links_ok and transactions_ok

def run_vote(block, validators, needed=3):
    votes = {name: validator_approves(v, block) for name, v in validators.items()}
    yes = sum(votes.values())
    for name, approved in votes.items():
        if not validators[name]["online"]:
            answer = "no answer (offline)"
        else:
            answer = "approve" if approved else "reject"
        print(f"   {name:<11} {answer}")
    decision = "ACCEPTED" if yes >= needed else "REJECTED"
```

```
        print(f"   -> {yes} of {len(votes)} approve: block {decision}")
        return decision
```

Case 1. GenoLab's server is down, the three others check block 41.

```
case_1 = {**validators, "GenoLab": {**validators["GenoLab"], "online": False}}
print("Case 1: honest block 41, one validator offline")
decision_1 = run_vote(block_41, case_1)
```

```
Case 1: honest block 41, one validator offline
   FreshMeals  approve
   Retailer A  approve
   Retailer B  approve
   GenoLab     no answer (offline)
   -> 3 of 4 approve: block ACCEPTED
```

Case 2. Someone proposes the relabelled block (BEEF). One validator, Retailer B, has been compromised and accepts the fake record.

```
fake_transaction = record_hash(relabelled_record)
case_2 = {
    **validators,
    "Retailer B": {**validators["Retailer B"], "known_transactions": original_transactions | {fake_transaction}}
}
print("Case 2: tampered block 41, one dishonest validator")
decision_2 = run_vote(tampered_block, case_2)
```

```
Case 2: tampered block 41, one dishonest validator
   FreshMeals  reject
   Retailer A  reject
   Retailer B  approve
   GenoLab     reject
   -> 1 of 4 approve: block REJECTED
```

Interpretation. With 4 validators and a 3-of-4 rule, the network still works if one member is offline (case 1), and one dishonest member cannot push a fake record (case 2). No mining (PoW) or tokens (PoS) are needed, because the members are known and bound by contract.

6. What this prototype proves, and what still needs production engineering

It proves:

- A SHA-256 fingerprint detects any change to an approved record (HORSE to BEEF), and a changed record breaks the block chain.
- One function can apply the business rules: role permissions, separation of duties and revocation, with two actions allowed and three denied.
- *(Optional)* A signature identifies who issued a record; a 3-of-4 vote tolerates one faulty or dishonest validator.

It does not prove, and still needs engineering:

- Real identities: company certificates, named accounts, two-factor login.
- A real permissioned network (for example Hyperledger Fabric) with 4 validator nodes and a consensus protocol.
- A web portal for suppliers, integration with each firm's ERP, off-chain storage for lab reports and prices.
- Security audit, monitoring, backups, and the legal charter of the consortium.
- **Truth of the physical world:** no code can see inside the meat. Independent DNA tests stay essential.